

Corte 3

Usuarios, Roles y Permisos

en PostgreSQL



Base de Datos Avanzadas · UPCh 2026

Ing. Ramsés Camas Nájera

Seguridad

GRANT / REVOKE

Roles

Multi-tenant

¿Qué veremos hoy?

01

¿Por qué importa la seguridad en BD?

El principio de mínimo privilegio

02

Usuarios y Roles

CREATE USER, CREATE ROLE, atributos, SET ROLE

03

Jerarquía y herencia de roles

GRANT role TO role, herencia automática

04

Los 5 niveles de permisos

Database → Schema → Tabla → Columna → Función

05

GRANT / REVOKE avanzado

Secuencias, funciones, DEFAULT PRIVILEGES

06

Auditoría y consulta de permisos

pg_roles, information_schema, psql shortcuts

07

Errores comunes y buenas prácticas

Los errores que todos cometen la primera vez

08

Caso práctico + Ejercicio

Sistema Hospital — 3 niveles desde cero

¿Por qué importa la seguridad en BD?

Un solo error de permisos puede costar millones

83%

de brechas de datos involucran
credenciales comprometidas

\$4.5M

costo promedio de una
brecha de datos (IBM, 2023)

Día 0

el momento en que debes
configurar permisos, no después

Principio de Mínimo Privilegio

"Cada usuario debe tener únicamente los permisos necesarios para realizar su trabajo, nada más."

✗ Mal: GRANT ALL TO PUBLIC

Cualquiera puede ver, modificar o borrar todo.

✓ Bien: GRANT SELECT TO lector

Solo quien necesita leer, puede leer.

¿Usuario o Rol? La confusión más común

En PostgreSQL, TODO es un ROLE internamente



CREATE USER = CREATE ROLE WITH LOGIN — son exactamente equivalentes

CREATE USER

- Tiene LOGIN por defecto
- Puede conectarse a la BD
- Lleva contraseña (PASSWORD)
- Se ve en pg_user
- Representa una persona real
- Ej: juan, ana, app_backend

CREATE ROLE

- Sin LOGIN por defecto
- No puede conectarse solo
- Actúa como grupo de permisos
- Se ve en pg_roles
- Representa un conjunto de permisos
- Ej: lector, editor, admin

CREATE USER — Sintaxis Completa

```
-- Forma básica
CREATE USER juan;

-- Con contraseña
CREATE USER juan WITH PASSWORD 'segura_2024!';

-- Con fecha de expiración
CREATE USER juan WITH PASSWORD 'pass' VALID UNTIL '2027-01-01';

-- Con atributos múltiples
CREATE USER dev_backend
  WITH PASSWORD 'backend_pass'
  CREATEDB
  VALID UNTIL '2028-06-01';

-- Usuario solo para conexión de app (sin contraseña interactiva)
CREATE USER app_user WITH PASSWORD 'app_secret' NOINHERIT;

-- Ver resultado
SELECT username, usesuper, usecreatedb, valuntil
FROM pg_user
WHERE username = 'juan';
```

Contraseñas fuertes

Mínimo 12 caracteres, combina mayúsculas, números y símbolos.

VALID UNTIL

Ideal para usuarios temporales o pasantes. Expira automáticamente.

NOINHERIT

Útil para cuentas de aplicación que no deben heredar permisos accidentalmente.

pg_user vs pg_roles

pg_user solo muestra roles con LOGIN. pg_roles muestra todos.

CREATE ROLE — Sintaxis y Atributos Clave

```
-- Rol sin LOGIN (grupo de permisos)
CREATE ROLE lector;
CREATE ROLE editor;
CREATE ROLE administrador;

-- Rol con LOGIN (equivale a CREATE USER)
CREATE ROLE juan WITH LOGIN PASSWORD 'pass';

-- Rol que puede crear otros roles (no superusuario)
CREATE ROLE team_lead WITH CREATEROLE;

-- Rol que NO hereda automáticamente (debe hacer SET ROLE)
CREATE ROLE auditoria WITH NOINHERIT;

-- Ver todos los roles
SELECT rolname, rolsuper, rolcreatedb,
       rolcreaterole, rolcanlogin, rolinherit
FROM pg_roles
ORDER BY rolname;
```

SUPERUSER

Omite TODAS las verificaciones.
Solo para DBA de confianza.

CREATEDB

Puede crear nuevas bases de datos.
Útil para devs en staging.

CREATEROLE

Puede crear roles sin ser superuser.
Ideal para team leads.

INHERIT

Hereda permisos de roles asignados.
Activado por defecto.

NOINHERIT

No hereda automáticamente.
Requiere SET ROLE explícito.

CONNECTION LIMIT

Limita conexiones concurrentes.
Ej: LIMIT 5 para apps.

SET ROLE y RESET ROLE

Cambiar de contexto de permisos dentro de una sesión

Cuando un usuario tiene asignado un rol con NOINHERIT, los permisos NO son automáticos. Debe ejecutar SET ROLE explícitamente para activarlos — y RESET ROLE para volver a su estado original.

```
-- Ejemplo: usuario 'ana' tiene NOINHERIT
-- y pertenece al rol 'auditoria'

-- 1. Ana se conecta — NO tiene permisos de auditoria aún
PSET: \c empresa ana

-- 2. Ana activa el rol de auditoria
SET ROLE auditoria;

-- 3. Ahora puede acceder a tablas de auditoria
SELECT * FROM log_accesos;

-- 4. Vuelve a sus permisos originales
RESET ROLE;
-- equivalente a: SET ROLE ana;

-- Ver el rol actual en sesión
SELECT current_user,    -- usuario de login
       session_user,    -- usuario de la sesión
       current_role;     -- rol activo ahora
```

¿Cuándo usar NOINHERIT?

Cuando quieres que un usuario tenga acceso a permisos sensibles SOLO de forma explícita y consciente.

Ej: cuentas de auditoría, roles de borrado masivo, roles de backup.

current_user vs session_user

session_user = quien se conectó originalmente

current_user = quien está activo ahora (cambia con SET ROLE)

Útil para logs de auditoría.

Jerarquía de Roles

Los roles pueden heredar de otros roles — como clases en OOP



```
-- Asignar roles a usuarios
GRANT lector TO juan, maría;
GRANT editor TO ana, pedro;
GRANT administrador TO root_user;

-- Herencia entre roles
GRANT lector TO editor;           -- editor hereda permisos de lector
GRANT editor TO administrador;   -- admin hereda todo lo anterior
```


INHERIT vs NOINHERIT — Detalle

No toda herencia es automática

```
-- INHERIT (comportamiento por defecto)
-- Los permisos del rol se activan automáticamente

CREATE ROLE lector;           -- rol base
CREATE USER ana WITH LOGIN INHERIT;  -- hereda auto
GRANT lector TO ana;

-- Ana ya puede usar permisos de 'lector' sin hacer nada más

-- -----
-- NOINHERIT – acceso manual y consciente
-- Los permisos NO se activan solos; requieren SET ROLE

CREATE ROLE auditoria;       -- rol sensible
CREATE USER inspector WITH NOINHERIT;  -- NO hereda auto
GRANT auditoria TO inspector;

-- inspector NO puede ver tablas de auditoria todavía
-- Debe ejecutar:
```



NOINHERIT es ideal para roles de auditoría, superacceso de emergencia, o cualquier permiso que deba quedar registrado en logs como acción deliberada.

ALTER ROLE y DROP ROLE

Modificar y eliminar roles de forma segura

```
-- Cambiar contraseña
ALTER USER dev_juan WITH PASSWORD 'nueva_pass_2024!';

-- Actualizar expiración
ALTER ROLE dev_juan VALID UNTIL '2028-01-01';

-- Agregar capacidad de crear DBs
ALTER ROLE dev_juan CREATEDB;

-- Quitarla
ALTER ROLE dev_juan NOCREATEDB;

-- Renombrar
ALTER ROLE dev_juan RENAME TO juan_dev;

-- Limitar conexiones simultáneas
ALTER ROLE app_user CONNECTION LIMIT 10;

-- Deshabilitar login sin borrar el usuario
ALTER ROLE juan NOLOGIN;
```



DROP ROLE — Precaución

```
-- Solo si NO tiene objetos
DROP ROLE lector;

-- Primero reasignar
REASSIGN OWNED
  BY lector TO postgres;
DROP OWNED BY lector;
DROP ROLE lector;
```



Buenas Prácticas

- Nunca DROP sin REASSIGN
- Documenta cada rol
- Audita pg_roles mensualmente
- Prefiere NOLOGIN sobre DROP cuando el usuario es temporal
- Usa CONNECTION LIMIT en cuentas de app

Los 5 Niveles de Permisos

De más amplio a más específico — todos son necesarios

Ejemplo

1	DATABASE CONNECT, CREATE	GRANT CONNECT ON DATABASE tienda TO lector;
2	SCHEMA USAGE (¡obligatorio para ver objetos!)	GRANT USAGE ON SCHEMA public TO lector;
3	TABLE SELECT, INSERT, UPDATE, DELETE, TRUNCATE	GRANT SELECT ON TABLE productos TO lector;
4	COLUMN SELECT / UPDATE en columnas específicas	GRANT SELECT (nombre, precio) ON productos TO lector;
5	FUNCTION EXECUTE — llamar funciones y procedures	GRANT EXECUTE ON FUNCTION calcular_total() TO editor;

El Error Más Frecuente: Olvidar el SCHEMA

GRANT SELECT en tablas NO funciona sin GRANT USAGE en el schema

✗ Situación real

"Le di SELECT a juan en todas las tablas pero me dice ERROR: permission denied for table productos"

```
-- ✗ INCOMPLETO – falta el schema
GRANT SELECT ON ALL TABLES IN SCHEMA public TO juan;
-- Juan sigue sin poder hacer SELECT → ERROR

-- ✓ CORRECTO – secuencia obligatoria
-- Paso 1: acceso a la base de datos
GRANT CONNECT ON DATABASE tienda TO juan;

-- Paso 2: acceso al schema (¡este es el que falta!)
GRANT USAGE ON SCHEMA public TO juan;

-- Paso 3: ahora sí, permisos sobre tablas
GRANT SELECT ON ALL TABLES IN SCHEMA public TO juan;

-- Ahora juan puede hacer SELECT sin errores ✓
```

GRANT — Niveles Database y Schema

-- ══ NIVEL DATABASE ══

-- Permitir conexión

```
GRANT CONNECT ON DATABASE tienda TO lector;
```

```
GRANT CONNECT ON DATABASE tienda TO editor;
```

-- Permitir crear schemas dentro de la BD

```
GRANT CREATE ON DATABASE tienda TO administrador;
```

-- Crear base de datos temporal

```
GRANT TEMPORARY ON DATABASE tienda TO lector;
```

-- ══ NIVEL SCHEMA ══

-- USAGE: permite ver/usar objetos del schema (REQUERIDO)

```
GRANT USAGE ON SCHEMA public TO lector;
```

```
GRANT USAGE ON SCHEMA public TO editor;
```

-- CREATE: permite crear tablas/funciones dentro del schema

```
GRANT CREATE ON SCHEMA public TO administrador;
```

-- Múltiples schemas

```
GRANT USAGE ON SCHEMA ventas, compras, rrhh TO editor;
```



Jerarquía obligatoria

Para que un usuario pueda hacer SELECT en una tabla, necesita:

- ① CONNECT en la BD
- ② USAGE en el schema
- ③ SELECT en la tabla

Si falta cualquiera → ERROR

Schemas múltiples

Una BD puede tener varios schemas:

- public (por defecto)
- ventas
- rrhh
- auditoria

Cada schema tiene sus propios permisos independientes.

Grant USAGE por schema, no solo en public.

GRANT — Tablas y Columnas

```
-- ══ NIVEL TABLA ══
```

```
-- Permisos específicos
```

```
GRANT SELECT ON TABLE productos TO lector;  
GRANT SELECT, INSERT ON TABLE pedidos TO editor;  
GRANT ALL PRIVILEGES ON TABLE empleados TO administrador;
```

```
-- Todas las tablas del schema
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lector;
```

```
-- Todas las vistas también
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lector;  
-- (las VIEWS son tablas a nivel de permisos)
```

```
-- ══ NIVEL COLUMNA ══
```

```
-- Útil cuando hay columnas sensibles (salarios, contraseñas)
```

```
GRANT SELECT (id, nombre, email) ON TABLE empleados TO lector;  
GRANT UPDATE (salario) ON TABLE empleados TO rrhh;
```

```
-- Ver qué columnas puede ver un rol
```

```
SELECT column_name, privilege_type  
FROM information_schema.column_privileges  
WHERE grantee = 'lector' AND table_name = 'empleados';
```

Permisos disponibles en TABLE

```
SELECT  
INSERT  
UPDATE  
DELETE  
TRUNCATE  
REFERENCES  
TRIGGER
```

ALL PRIVILEGES = todos los anteriores

¿Por qué permisos de columna?

Caso real: tabla 'empleados' con columna 'salario'.

Lector puede ver: id, nombre, email.
RRHH puede ver Y actualizar: salario.
Nadie más toca salario.

GRANT — Funciones, Procedures y Secuencias

```
-- ══ FUNCIONES Y STORED PROCEDURES ══
```

```
GRANT EXECUTE ON FUNCTION calcular_total(INT) TO editor;  
GRANT EXECUTE ON FUNCTION generar_reporte() TO lector;
```

```
-- Todas las funciones del schema
```

```
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO editor;  
GRANT EXECUTE ON ALL PROCEDURES IN SCHEMA public TO editor;
```

```
-- ══ SECUENCIAS (auto-increment IDs) ══
```

```
-- Sin esto, INSERT falla aunque tengas permiso en la tabla
```

```
GRANT USAGE ON SEQUENCE pedidos_id_seq TO editor;  
GRANT SELECT ON SEQUENCE pedidos_id_seq TO lector;  
GRANT UPDATE ON SEQUENCE pedidos_id_seq TO editor;
```

```
-- Forma corta: USAGE incluye SELECT y UPDATE para sequences  
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO editor;
```

```
-- Verificar secuencias que tiene un rol  
SELECT sequence_name, privilege_type  
FROM information_schema.usage_privileges  
WHERE grantee = 'editor';
```

Error con INSERT y secuencias

Si das GRANT INSERT en una tabla con columna SERIAL o BIGSERIAL y no das GRANT USAGE en la secuencia correspondiente:

ERROR: permission denied
for sequence pedidos_id_seq

Naming de secuencias

PostgreSQL nombra las secuencias así:

{tabla}_{columna}_seq

Ejemplos:

- pedidos_id_seq
- empleados_id_seq
- productos_codigo_seq

Verificalas con: \ds en psql

REVOKE — Revocar Permisos

El complemento de GRANT — igual de crítico

```
-- Sintaxis general
REVOKE <privilegio> ON <objeto> FROM <rol>;

-- Quitar SELECT de una tabla
REVOKE SELECT ON TABLE productos FROM lector;

-- Quitar todos los privilegios
REVOKE ALL PRIVILEGES ON TABLE productos FROM editor;

-- Quitar acceso a la base de datos
REVOKE CONNECT ON DATABASE tienda FROM lector;

-- Revocar GRANT OPTION (sin quitar el permiso base)
-- (Ana podía reenviar permisos a otros — ya no puede)
REVOKE GRANT OPTION FOR SELECT ON TABLE ventas FROM ana;

-- Revocación en cascada
-- Quita permisos que Ana había otorgado a terceros también
REVOKE SELECT ON TABLE ventas FROM ana CASCADE;

-- Revocar rol de un usuario
REVOKE lector FROM juan;

-- Revocar en múltiples usuarios
REVOKE editor FROM ana, pedro;
```

CASCADE — Cuidado

Cuando revocas con CASCADE también revoca los permisos que ese rol había otorgado a terceros.

Ejemplo:

Ana tenía GRANT OPTION → se lo dio a carlos
Al revocar de Ana con CASCADE → carlos también pierde el permiso

Antes de REVOKE — audita

Siempre consulta los permisos actuales antes de revocar:

\dp nombre_tabla

o:
SELECT * FROM
information_schema.
role_table_grants
WHERE grantee='ana';

ALTER DEFAULT PRIVILEGES

El error más común: GRANT no cubre tablas futuras

✗ El Problema Clásico

Hiciste GRANT en las tablas de hoy. Mañana creas una nueva tabla → lector NO tiene acceso. Hay que volver a hacer GRANT.

-- ✓ La solución: DEFAULT PRIVILEGES

-- Permisos automáticos en TODOS los objetos futuros

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT ON TABLES TO lector;
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO editor;
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ALL PRIVILEGES ON TABLES TO administrador;
```

-- Para secuencias futuras (importante para INSERT)

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT USAGE, SELECT ON SEQUENCES TO editor;
```

-- Para funciones futuras

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT EXECUTE ON FUNCTIONS TO editor;
```

-- Ver los default privileges configurados

```
SELECT * FROM pg_default_acl;
```

Consultar Permisos (1/2) — pg_roles y pg_auth_members

```
-- — VER TODOS LOS ROLES —
SELECT rolname,
       rolsuper      AS superuser,
       rolcreatedb   AS crea_bd,
       rolcanlogin   AS puede_login,
       rolvaliduntil AS expira
FROM pg_roles
ORDER BY rolname;

-- — VER A QUÉ ROLES PERTENECE CADA USUARIO —
SELECT r.rolname AS miembro,
       m.rolname AS pertenece_a,
       a.admin_option AS es_admin_del_rol
FROM pg_auth_members a
JOIN pg_roles r ON r.oid = a.member
JOIN pg_roles m ON m.oid = a.roleid
ORDER BY r.rolname;

-- — ROLES DE UN USUARIO ESPECÍFICO —
SELECT r.rolname
FROM pg_auth_members am
JOIN pg_roles r ON r.oid = am.roleid
WHERE am.member = (
    SELECT oid FROM pg_roles WHERE rolname = 'ana'
);
```

Comandos `psql`

`\du`

Lista todos los roles y atributos

`\du+`

Lista roles con detalles extra

`\dp tabla`

Permisos de una tabla (Access Privileges)

`\dp`

Permisos de todas las tablas

`\l`

Lista de bases de datos con permisos

`\dn+`

Schemas con sus permisos

`\df+`

Funciones con sus permisos

Consultar Permisos (2/2) — information_schema

```
-- — PERMISOS SOBRE TABLAS —
```

```
SELECT grantee,  
       table_name,  
       privilege_type,  
       is_grantable  
FROM information_schema.role_table_grants  
WHERE table_schema = 'public'  
ORDER BY grantee, table_name;
```

```
-- — PERMISOS SOBRE COLUMNAS —
```

```
SELECT grantee, table_name, column_name, privilege_type  
FROM information_schema.column_privileges  
WHERE table_schema = 'public'  
ORDER BY grantee, table_name, column_name;
```

```
-- — TODOS LOS PRIVILEGIOS DE UN ROL —
```

```
SELECT object_type, object_name, privilege_type  
FROM (  
  SELECT 'TABLE' AS object_type, table_name AS object_name, privilege_type  
  FROM information_schema.role_table_grants  
  WHERE grantee = 'editor'  
  UNION ALL  
  SELECT 'ROUTINE', routine_name, privilege_type  
  FROM information_schema.role_routine_grants  
  WHERE grantee = 'editor'  
) permisos
```

```
ORDER BY object_type, object_name;
```

Errores Comunes — y Cómo Resolverlos

Los 5 errores que todos cometen la primera vez

permission denied for table X

Causa: Diste SELECT en la tabla pero olvidaste GRANT USAGE ON SCHEMA

Fix: `GRANT USAGE ON SCHEMA public TO rol;`

permission denied for sequence X

Causa: Diste INSERT en la tabla pero olvidaste la secuencia del ID

Fix: `GRANT USAGE ON SEQUENCE tabla_id_seq TO rol;`

permission denied for database X

Causa: El usuario existe pero no tiene CONNECT en la BD

Fix: `GRANT CONNECT ON DATABASE mi_bd TO rol;`

Nueva tabla no accesible

Causa: DEFAULT PRIVILEGES no configurado — GRANT no cubre tablas futuras

Fix: `ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ... TO rol;`

DROP ROLE falla

Causa: El rol tiene objetos o dependencias — no se puede borrar directamente

Fix: `REASSIGN OWNED BY rol TO postgres;
DROP OWNED BY rol; DROP ROLE rol;`

Buenas Prácticas de Seguridad

Los estándares que usa la industria

Nunca usar el usuario 'postgres' en apps

Crea un usuario dedicado con solo los permisos que necesita. 'postgres' es el superusuario — es como usar root en Linux para todo.

Nombrar roles con su función, no con nombres de personas

BIEN: lector, editor, api_backend, auditoria
MAL: juan, el_jefe, mis_permisos

Usar VALID UNTIL para accesos temporales

Pasantes, proyectos, usuarios de prueba — con fecha de expiración automática nunca quedan cuentas fantasma.

Auditar permisos mensualmente

Revisar pg_roles, pg_auth_members e information_schema. Eliminar roles obsoletos. Detectar privilegios no autorizados.

Separar usuario de app del usuario de admin

La app usa app_user con SELECT/INSERT/UPDATE. Las migraciones usan db_admin. Nunca el mismo usuario para todo.

Documentar cada rol creado

Un comentario en la BD es suficiente: COMMENT ON ROLE lector IS 'Solo lectura en tablas públicas';

Resumen: El Flujo Completo de Seguridad

El orden correcto para configurar una BD desde cero

- 1 Crear ROLES sin LOGIN como grupos de permisos
- 2 Crear USERS con LOGIN para personas y aplicaciones
- 3 GRANT CONNECT a la base de datos
- 4 GRANT USAGE al schema — ¡nunca olvidar este paso!
- 5 GRANT permisos específicos sobre tablas, columnas y funciones
- 6 GRANT USAGE en secuencias si los roles van a hacer INSERT
- 7 ALTER DEFAULT PRIVILEGES para que aplique también a objetos futuros
- 8 GRANT rol TO usuario para asignar grupos a personas reales

Caso Práctico: Sistema Empresa — Diseño

Antes de codificar, siempre diseña la matriz de permisos

Rol	productos	ventas	empleados	salarios	reportes
lector	SELECT	SELECT	SELECT (id,nombre)	—	SELECT
editor	SELECT INSERT UPDATE	SELECT INSERT	SELECT	—	SELECT
rrhh	SELECT	—	SELECT UPDATE	UPDATE	SELECT
administrador	ALL	ALL	ALL	ALL	ALL

Antes de escribir una sola línea de SQL, define quién puede hacer qué. Esta matriz es tu contrato de seguridad.

Caso Práctico: Sistema Empresa — Código

① Roles
Implementación completa del diseño anterior

```
CREATE ROLE lector;  
CREATE ROLE editor;  
CREATE ROLE rrhh;  
CREATE ROLE administrador;  
  
-- ② Herencia  
GRANT lector TO editor;  
GRANT lector TO rrhh;  
  
-- ③ Acceso a BD y schema  
GRANT CONNECT ON DATABASE empresa TO lector,editor,rrhh,administrador;  
GRANT USAGE ON SCHEMA public TO lector,editor,rrhh,administrador;  
  
-- ④ Tablas  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lector;  
GRANT INSERT, UPDATE ON productos, ventas TO editor;  
GRANT UPDATE ON empleados TO rrhh;  
GRANT UPDATE (salario) ON empleados TO rrhh;  
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO administrador;  
  
-- ⑤ Secuencias  
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO editor;  
  
-- ⑥ Default privileges  
ALTER DEFAULT PRIVILEGES IN SCHEMA public  
GRANT SELECT ON TABLES TO lector;  
  
-- ⑦ Usuarios  
CREATE USER juan WITH PASSWORD 'pass1'; GRANT lector TO juan;  
CREATE USER ana WITH PASSWORD 'pass2'; GRANT editor TO ana;
```


Ejercicio en Clase

30 minutos — Diseña e implementa

Escenario: Sistema de Hospital

Tablas: pacientes · citas · medicamentos · empleados · historial_clinico · facturas · recetas

Usuarios reales: *dra_lopez*, *enfermero_juan*, *repcionista_maria*, *director_rrhh*, *sistema_app*

A Diseña la matriz de permisos (en papel/código)

B Crea los roles: enfermero, medico, recepcionista, director, app

C medico hereda permisos de enfermero

D enfermero: SELECT en pacientes y citas; INSERT en historial

E medico: agrega INSERT/UPDATE en historial y recetas

F director: ALL en tablas excepto historial_clinico

G app: solo puede ejecutar stored procedures (EXECUTE), no SELECT directo

H Configura DEFAULT PRIVILEGES para todas las tablas futuras



Seguridad por capas

"La seguridad no es un add-on — es parte del diseño desde el día cero."

CREATE ROLE

GRANT

REVOKE

DEFAULT PRIVILEGES

pg_roles

SET ROLE



Próximo tema: Row-Level Security (RLS) — Políticas por fila